

VOLUME 3

From index.html to a Living Interface

*React startup, effects, lazy loading, responsive shell selection, errors,
and composition*

A beginner-to-engineer rebuild guide grounded in the real repository

PATTERN > PRACTICE > REPETITION > REFINEMENT > HABIT

Learning Map

Use this volume as a build session, not passive reading. Type the examples, pause at checkpoints, and keep the architecture questions beside your editor.

Chapter	You will learn
1. Browser-to-React startup	Trace execution from HTML to App.
2. App composition	Understand the startup effect and ready state.
3. Responsive shells	Choose desktop or mobile behavior without mixing them.
4. Lazy loading and Suspense	Load large app windows only when needed.
5. Error and loading boundaries	Keep failures understandable and local.

Chapter 1: Browser-to-React Startup

Chapter outcome: You will explain exactly what runs before the first Macfolio screen appears.

Startup execution flow

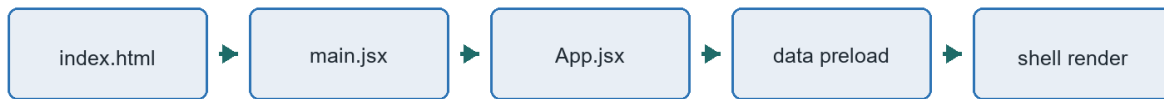


Figure 1. Startup is a sequence of boundaries, not magic.

The browser document

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/macbook.png" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Aditya's Portfolio</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```

The React mount

```

import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './app/App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)

```

Vite serves index.html and follows the module script to src/main.jsx. The CSS import becomes a stylesheet in the bundle. React then owns the #root element; components return descriptions of UI, and React synchronizes those descriptions with the DOM.

Chapter 2: App Composition

Chapter outcome: You will read the real App component as a coordinator and understand every stage of its startup effect.

Startup state and effect

```

const ContactWindow = lazy(() => import("../windows/Contact"));
const Gallery = lazy(() => import("../windows/Gallery"));
const VsCode = lazy(() => import("../windows/VsCode"));
const MobileApp = lazy(() => import("../features/mobile-shell/MobileApp"));
const Welcome = lazy(() => import("../components/Welcome"));

const App = () => {
  const isMobile = useMediaQuery({ maxWidth: 768 });
  const { windows } = useWindowStore();
  const { setActiveLocation } = useLocationStore();
  const { loadPortfolioFileSystem } = usePortfolioFileSystem();

  const [isReady, setIsReady] = useState(false);

  useEffect(() => {
    let isAlive = true;
    let readyTimer = null;

    const preload = async () => {
      try {
        const work = await loadPortfolioFileSystem();

        if (isAlive) {
          setActiveLocation(work);
        }
      } finally {
        readyTimer = setTimeout(() => {
          if (isAlive) {
            setIsReady(true);
          }
        }, 2200);
      }
    };

    preload();

    return () => {
      isAlive = false;
      if (readyTimer) {
        clearTimeout(readyTimer);
      }
    };
  }, [loadPortfolioFileSystem, setActiveLocation]);

  if (!isReady) return <IntroLoader />;

  if (isMobile) {
    return (

```

- useMediaQuery subscribes to viewport width.
- The window store provides desktop window records.
- The location store receives the initial Work folder.
- usePortfolioFileSystem hides loading and mapping details.
- isReady keeps the intro visible for at least the intended presentation delay.
- The cleanup flag prevents state updates after unmount; the timer is also cleared.

Complex function explained: useEffect runs after React commits the component. Its dependency array says the effect should be recreated if either referenced function identity changes. The returned

cleanup runs before recreation and on unmount, protecting asynchronous work from updating a component that no longer exists.

The finally block is deliberate: even if remote GitHub data fails, the app leaves the intro screen and can continue with fallback or empty data. Reliability sometimes means showing a useful interface while a non-critical service is unavailable.

Chapter 3: Responsive Shells

Chapter outcome: You will separate a shared product from two interaction models.

Shell selection and desktop registrations

```

    <Suspense fallback={<IntroLoader />}>
      <MobileApp />
    </Suspense>
  </ErrorBoundary>
);
}

const desktopRenderWindow = (Component, id) => {
  const win = windows[id];

  if (!win?.isOpen || win?.isMinimized) {
    return null;
  }

  return <Component key={id} windowId={id} />;
};

return (
  <ErrorBoundary>
    <main className="relative overflow-hidden">
      <Navbar />
      <Suspense fallback={null}>
        <Welcome />
      </Suspense>
      <Dock />

      <ErrorBoundary>
        <Suspense fallback={null}>
          <div className="window-layer">
            {desktopRenderWindow(Terminal, "terminal")}
            {desktopRenderWindow(Safari, "safari")}
            {desktopRenderWindow(Resume, "resume")}
            {desktopRenderWindow(Finder, "finder")}
            {desktopRenderWindow(TextWindow, "txtfile")}
            {desktopRenderWindow(ImageWindow, "imgfile")}
            {desktopRenderWindow(ContactWindow, "contact")}
            {desktopRenderWindow(Gallery, "photos")}
            {desktopRenderWindow(VsCode, "vsCode")}
          </div>
        </Suspense>
      </ErrorBoundary>

      <Home />
    </main>
  </ErrorBoundary>
);

```

```

    </ErrorBoundary>
  );
};

export default App;

```

Desktop and mobile are not just different CSS widths. Desktop uses overlapping windows, z-index, drag, and a Dock. Mobile uses a stack of full-screen app routes and a back action. Keeping the shells separate lets each interaction model remain coherent while shared portfolio concepts can stay below them.

When not to split: If mobile only rearranges the same page sections, responsive CSS may be enough. Split shells when navigation, state transitions, and interaction rules differ materially.

Chapter 4: Lazy Loading and Suspense

Chapter outcome: You will understand bundle splitting and know which components deserve lazy loading.

The lazy window pattern

```

const Finder = lazy(() => import('../windows/Finder'));

<Suspense fallback={null}>
  {desktopRenderWindow(Finder, 'finder')}
</Suspense>

```

WHAT React.lazy turns a dynamic import into a component whose code is requested on demand.

WHY Large editors, PDF renderers, galleries, and secondary apps should not delay the initial screen.

WHEN Use it for substantial routes or app surfaces that may never open in a session.

WHERE Keep declarations near the app or route registry that chooses the component.

HOW Wrap lazy components in Suspense and provide a fallback appropriate to the surrounding interface.

Tradeoff: Too many tiny chunks add network overhead and make debugging noisier. Lazy-load meaningful feature surfaces, not every button.

Chapter 5: Error and Loading Boundaries

Chapter outcome: You will design failure states that protect the rest of the experience.

Class error boundary used by Macfolio

```

import React from "react";

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);

    this.state = {
      hasError: false,
      error: null,
    };
  }
}

```

```

        errorInfo: null,
      };
    }

    static getDerivedStateFromError(error) {
      return {
        hasError: true,
        error,
      };
    }

    componentDidCatch(error, errorInfo) {
      console.error("Caught by ErrorBoundary:", error, errorInfo);

      this.setState({
        error,
        errorInfo: errorInfo || null,
      });
    }

    render() {
      if (this.state.hasError) {
        return (
          <div
            style={{
              width: "100vw",
              height: "100vh",
              background: "#111",
              color: "white",
              padding: "30px",
              overflow: "auto",
              fontFamily: "monospace",
            }}
          >
            <h2>App Crashed</h2>

            <pre style={{ whiteSpace: "pre-wrap" }}>
              {this.state.error?.toString()}
            </pre>

            <pre style={{ whiteSpace: "pre-wrap", opacity: 0.7 }}>
              {this.state.errorInfo?.componentStack || "No component stack available"}
            </pre>
          </div>
        );
      }

      return this.props.children;
    }
  }

  export default ErrorBoundary;

```

React error boundaries catch rendering and lifecycle errors below them. They do not catch event-handler errors, asynchronous promise rejections, or server failures by themselves. Macfolio uses an outer boundary for the whole shell and another around desktop windows so one broken app surface does not necessarily destroy the desktop.

Checkpoint

- Beginner: Draw the first five startup steps from memory.
- Intermediate: Explain why the Work folder is loaded before desktop Finder opens.
- Advanced: Choose boundaries for an app with dashboard, billing, and a chat widget.